

---

## CSA0918- JAVA PROGRAMMING DESIGN TASK

---

NAME: J O JOANNA DIVINE

REG NO: 192572003

---

### 1. Problem Understanding and Formulation

The project addresses a production-style concurrency problem in a ticketing platform similar to BookMyShow or Ticketmaster. During a flash sale, many customers may attempt to book the same seat within milliseconds. If the booking code checks whether a seat is available and updates its state as separate non-atomic operations, multiple threads can observe the seat as available and all proceed to create bookings. This produces double-allotment, refunds, and loss of customer trust.

The objective of SeatLock is to model a ticket reservation system in Java and prove that the booking core remains correct under concurrent access. The system deliberately demonstrates the unsafe implementation first and then applies a thread-safe design using a per-show ReentrantLock.

The implementation uses a 100-seat concert inventory divided into VIP, Premium, and Regular categories. Concurrent tests use 50 and 100 customer threads. Payment is simulated because a real payment gateway is outside the assignment scope.

### 2. Problem Statement

Design and implement a concurrent ticket reservation engine in Java that models shows, customers, bookings, payments, and categorized seats; maintains inventory using Java Collections; demonstrates a race condition using concurrent threads; fixes the race using appropriate synchronization; handles failures using custom checked exceptions; persists booking data through JDBC; maintains an audit log; and optionally provides a Swing dashboard for interactive seat selection.

### 3. Objectives

- Apply OOP principles including inheritance, polymorphism, abstraction, and encapsulation.
- Use the Java Collection Framework with generics and Iterator traversal.
- Reproduce a real check-then-act race condition under concurrent booking attempts.
- Guarantee zero duplicate seat allocations in the thread-safe implementation.
- Ensure payment failure releases a tentatively reserved seat.
- Persist bookings and cancellations using JDBC and parameterized SQL.
- Provide customer booking history and audit logging.
- Validate correctness through explicit invariants and repeatable stress tests.
- Provide an interactive Swing customer dashboard as an enhancement.

### 4. System Design and Architecture

The system is organized into domain, exception, service, database, UI, and main application layers.

| Layer / Package | Major Components                                                                                           | Purpose                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| model           | Seat, RegularSeat, PremiumSeat, VIPSeat, Show, Customer, Booking, Payment, SeatStatus                      | Domain model and business entities                           |
| exception       | SeatNotAvailableException, InvalidSeatSelectionException, PaymentFailureException, BookingTimeoutException | Explicit failure handling                                    |
| service         | BookingService, ConcurrencyMetrics, InvariantChecker, AuditLogger                                          | Booking logic, concurrency measurement, validation and audit |
| database        | DatabaseManager                                                                                            | SQLite persistence through JDBC                              |
| ui              | AdminDashboard                                                                                             | Interactive Swing ticketing dashboard                        |

|      |      |                                           |
|------|------|-------------------------------------------|
| main | Main | CLI menu and automated verification suite |
|------|------|-------------------------------------------|

## 5. OOP and Domain Modeling

### 5.1 Seat Hierarchy

Seat is the base abstraction. RegularSeat, PremiumSeat, and VIPSeat extend Seat and override calculatePrice(). This demonstrates inheritance and runtime polymorphism. The booking system can work with a Seat reference while the actual subclass determines the final price.

| Seat Type | Inventory | Final Price |
|-----------|-----------|-------------|
| VIP       | 1–20      | \$500.00    |
| Premium   | 21–50     | \$180.00    |
| Regular   | 51–100    | \$60.00     |

During verification, polymorphic calls produced RegularSeat = \$50.00, PremiumSeat = \$75.00, and VIPSeat = \$125.00 when using a demonstration base price of \$50.00. The production inventory uses the 100-seat pricing shown above.

### 5.2 Encapsulation

Domain data is kept inside classes rather than exposed as public mutable fields. Access is provided through appropriate methods. This prevents unrelated parts of the program, especially the UI, from directly changing booking state and bypassing business rules.

## 6. Collections Framework

The Show maintains seat inventory using Map<Integer, Seat>, providing direct lookup by seat number. Booking history is maintained using List<Booking>. Generics provide compile-time type safety. Iterator traversal is explicitly demonstrated in the verification suite.

| Structure          | Choice          | Reason                                             |
|--------------------|-----------------|----------------------------------------------------|
| Map<Integer, Seat> | Inventory       | Fast lookup of a seat by unique seat number        |
| List<Booking>      | Booking history | Maintains an ordered collection of booking records |

|                   |                   |                                                                              |
|-------------------|-------------------|------------------------------------------------------------------------------|
| Iterator<Booking> | History traversal | Demonstrates interface-based traversal without exposing collection internals |
|-------------------|-------------------|------------------------------------------------------------------------------|

HashMap is suitable for the single-JVM inventory because the main requirement is direct lookup by seat ID rather than sorted order. TreeMap would add ordering that is not necessary. ConcurrentHashMap can support concurrent map operations, but it does not by itself make a multi-step booking transaction atomic. The final design therefore uses a per-show ReentrantLock around the critical booking operation.

## 7. Concurrency Problem and Race Condition

The unsafe booking logic demonstrates the classic check-then-act race condition. Multiple customer threads can execute a sequence equivalent to: check whether the seat is AVAILABLE, pause or interleave with another thread, then reserve/book the seat. Because the complete sequence is not atomic, several threads can pass the availability check before any thread completes the update.

Before/after evidence:

| Test           | Users | Successful | Rejected | Duplicate Bookings | Result         |
|----------------|-------|------------|----------|--------------------|----------------|
| Unsynchronized | 50    | 19         | 31       | 18                 | RACE CONDITION |
| ReentrantLock  | 50    | 1          | 49       | 0                  | PASS           |
| ReentrantLock  | 100   | 1          | 99       | 0                  | PASS           |

The unsynchronized run is deliberately retained because it provides experimental evidence of the original production bug. The synchronized run demonstrates that the same contention scenario results in exactly one successful booking and zero duplicate allocations.

## 8. Synchronization Strategy

### 8.1 ReentrantLock

The final implementation uses a ReentrantLock associated with each Show. The critical booking operation acquires the show's lock before checking and changing seat state and releases it after

the transaction finishes. This makes the relevant check-update-payment sequence mutually exclusive for that show.

## 8.2 Why Per-Show Locking?

A single global lock would unnecessarily serialize unrelated shows. Per-show locking limits contention to customers booking the same show while allowing bookings for different shows to proceed independently. This provides a better balance between simplicity, correctness, and concurrency.

## 8.3 Alternative: synchronized

A synchronized method or block would also be valid for the assignment and is simple to understand. ReentrantLock was selected because it provides explicit lock management and timed acquisition through tryLock(), which supports the required BookingTimeoutException scenario.

## 9. Atomic Booking and Seat States

SeatLock models the important transaction states as AVAILABLE, RESERVED, and BOOKED. A successful booking follows the conceptual sequence AVAILABLE → RESERVED → payment success → BOOKED. A payment failure follows AVAILABLE → RESERVED → payment failure → AVAILABLE. The rollback prevents a seat from remaining locked but unpaid.

The GUI distinguishes these states using green for AVAILABLE, blue for SELECTED (a UI selection state), red for BOOKED, and yellow for PROCESSING/RESERVED. The GUI does not directly perform booking state changes; it delegates booking to BookingService.

## 10. Exception Handling

The project defines four custom checked exceptions:

| Exception                     | Purpose                                                              |
|-------------------------------|----------------------------------------------------------------------|
| SeatNotAvailableException     | Raised when a requested seat is already unavailable/booked.          |
| InvalidSeatSelectionException | Raised when a requested seat does not exist or selection is invalid. |

|                         |                                                                                                 |
|-------------------------|-------------------------------------------------------------------------------------------------|
| PaymentFailureException | Raised when the simulated payment fails and rollback is required.                               |
| BookingTimeoutException | Raised when a timed lock acquisition cannot obtain the show lock within the configured timeout. |

The verification suite explicitly catches all four exceptions. The BookingTimeoutException test uses a background thread to hold the show's lock while another thread attempts timed acquisition for 50 ms, causing tryLock() to fail and the custom exception to be thrown.

### 11. Payment Failure and Rollback

Payment gateway integration is simulated as permitted by the assignment. When payment is configured to fail, PaymentFailureException is raised after the seat has been tentatively reserved. The booking logic catches the failure and restores the seat to AVAILABLE. Verification output confirmed: Initial Seat #5 Status = AVAILABLE; PaymentFailureException caught; Post-failure Seat #5 Status = AVAILABLE.

### 12. JDBC Persistence

JDBC (Java Database Connectivity) provides the bridge between the Java application and the SQLite relational database. The project uses a SQLite database file named bookings.db. DatabaseManager handles persistence, transaction control, booking queries, and cancellation.

| JDBC Component    | Role                                                                     |
|-------------------|--------------------------------------------------------------------------|
| Connection        | Opens the Java-to-database connection and controls transaction behavior. |
| PreparedStatement | Executes parameterized SQL and avoids string-concatenated SQL.           |
| ResultSet         | Reads rows returned by SELECT queries.                                   |
| commit()          | Permanently saves successful database changes.                           |
| rollback()        | Reverts database changes when a transaction fails.                       |

PreparedStatement is used instead of concatenating customer-supplied values into SQL strings. Parameterization improves safety and prevents SQL injection patterns. The customer history feature uses a parameterized query by customer ID, and cancellation updates the booking status to CANCELLED.

### **13. JDBC Transaction Evidence**

The verification suite created booking SAFE-BKG-5253bb66 for Seat #3, queried the database, and then cancelled the booking. The database record changed to CANCELLED and Seat #3 returned to AVAILABLE. The persistence and cancellation verification completed successfully.

### **14. Java I/O and Audit Logging**

AuditLogger maintains a transaction audit log in booking\_audit.log. Logging records significant booking events and provides a trace that can be inspected after execution. Thread-safe logging is important because multiple customer threads may generate events concurrently.

### **15. Invariant-Based Validation**

The system validates the following consistency conditions:

- No duplicate seat allocations.
- Successful bookings do not exceed available inventory constraints.
- Failed payment seats are restored.
- No stale RESERVED seats remain after a completed transaction.
- Database consistency is verified.

The final verification suite reported ALL INVARIANTS SATISFIED.

# 16. Interactive Swing Dashboard

SeatLock - Concert Ticket Reservation & Concurrency Engine

SEATLOCK CONCERT HALL

Coldplay — Music of the Spheres World Tour

STAGE/MAIN SCREEN

● AVAILABLE ● SELECTED ● BOOKED ● PROCESSING / RESERVED

VIP SECTION • \$500.00

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| #1  | #2  | #3  | #4  | #5  | #6  | #7  | #8  | #9  | #10 |
| #11 | #12 | #13 | #14 | #15 | #16 | #17 | #18 | #19 | #20 |

PREMIUM SECTION • \$180.00

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| #21 | #22 | #23 | #24 | #25 | #26 | #27 | #28 | #29 | #30 |
| #31 | #32 | #33 | #34 | #35 | #36 | #37 | #38 | #39 | #40 |
| #41 | #42 | #43 | #44 | #45 | #46 | #47 | #48 | #49 | #50 |

REGULAR SECTION • \$60.00

|     |     |     |     |     |     |     |     |     |      |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|
| #51 | #52 | #53 | #54 | #55 | #56 | #57 | #58 | #59 | #60  |
| #61 | #62 | #63 | #64 | #65 | #66 | #67 | #68 | #69 | #70  |
| #71 | #72 | #73 | #74 | #75 | #76 | #77 | #78 | #79 | #80  |
| #81 | #82 | #83 | #84 | #85 | #86 | #87 | #88 | #89 | #90  |
| #91 | #92 | #93 | #94 | #95 | #96 | #97 | #98 | #99 | #100 |

TOTAL SEATS

100

AVAILABLE

100

SELECTED

6

RESERVED

0

BOOKED

0

Refresh Seating Map

YOUR BOOKING

Customer ID: 101

Customer Name: John Doe

Selected Seats

Seat #34 (Premium) - \$180.00  
Seat #38 (Premium) - \$180.00  
Seat #63 (Regular) - \$60.00  
Seat #87 (Regular) - \$60.00

TOTAL AMOUNT \$1480.00

Payment: ☐ Simulate Success ☒ Simulate Failure

BOOK SELECTED SEATS

Clear Selection

CONCURRENCY OBSERVATORY

Ready for stress testing...  
Select 50 or 100 users below.

Run 50-User Test

Run 100-User Test

SeatLock - Concert Ticket Reservation & Concurrency Engine

SEATLOCK CONCERT HALL

Coldplay — Music of the Spheres World Tour

STAGE/MAIN SCREEN

● AVAILABLE ● SELECTED ● BOOKED ● PROCESSING / RESERVED

VIP SECTION • \$500.00

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| #1  | #2  | #3  | #4  | #5  | #6  | #7  | #8  | #9  | #10 |
| #11 | #12 | #13 | #14 | #15 | #16 | #17 | #18 | #19 | #20 |

PREMIUM SECTION • \$180.00

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| #21 | #22 | #23 | #24 | #25 | #26 | #27 | #28 | #29 | #30 |
| #31 | #32 | #33 | #34 | #35 | #36 | #37 | #38 | #39 | #40 |
| #41 | #42 | #43 | #44 | #45 | #46 | #47 | #48 | #49 | #50 |

REGULAR SECTION • \$60.00

|     |     |     |     |     |     |     |     |     |      |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|------|
| #51 | #52 | #53 | #54 | #55 | #56 | #57 | #58 | #59 | #60  |
| #61 | #62 | #63 | #64 | #65 | #66 | #67 | #68 | #69 | #70  |
| #71 | #72 | #73 | #74 | #75 | #76 | #77 | #78 | #79 | #80  |
| #81 | #82 | #83 | #84 | #85 | #86 | #87 | #88 | #89 | #90  |
| #91 | #92 | #93 | #94 | #95 | #96 | #97 | #98 | #99 | #100 |

TOTAL SEATS

100

AVAILABLE

97

SELECTED

0

RESERVED

1

BOOKED

2

Refresh Seating Map

YOUR BOOKING

Customer ID: 101

Customer Name: John Doe

Selected Seats

(No seats selected)

TOTAL AMOUNT \$0.00

Payment: ☐ Simulate Success ☒ Simulate Failure

BOOK SELECTED SEATS

Clear Selection

PROCESSING PAYMENT... (State: RESERVED)

CONCURRENCY OBSERVATORY

Ready for stress testing...  
Select 50 or 100 users below.

Run 50-User Test

Run 100-User Test



SEATLOCK - Concert Ticket Reservation & Concurrency Engine

SEATLOCK CONCERT HALL

Coldplay — Music of the Spheres World Tour

STAGE/MAIN SCREEN

AVAILABLESELECTEDBOOKEDPROCESSING / RESERVED

VIP SECTION • \$500.00

#1#2#3#4#5#6#7#8#9#10

#11#12#13#14#15#16#17#18#19#20

PREMIUM SECTION • \$180.00

#21#22#23#24#25#26#27#28#29#30

#31#32#33#34#35#36#37#38#39#40

REGULAR SECTION • \$60.00

#41#42#43#44#45#46#47#48#49#50

#51#52#53#54#55#56#57#58#59#60

#61#62#63#64#65#66#67#68#69#70

#71#72#73#74#75#76#77#78#79#80

#81#82#83#84#85#86#87#88#89#90

#91#92#93#94#95#96#97#98#99#100

TOTAL SEATSTOTAL AVAILABLESELECTEDRESERVEDBOOKED

10094006

SEATLOCK

Concurrent Ticket Reservation Engine (100 Seats)

1. View Seat Inventory & OOP Polymorphism Demo

2. Run Race Condition Test (50 Users Unsynchronized)

3. Run Thread-Safe Stress Test (50 Users ReentrantLock)

4. Run Thread-Safe Stress Test (100 Users ReentrantLock)

5. Run Payment Failure & Rollback Test

6. Run JDBC Persistence & Cancellation Test

7. Run Full Verification Suite (All Tests)

8. View Customer Booking History

9. Launch Customer GUI Dashboard (Swing)

10. Exit

Select an option (1-10): 8

Enter Customer ID (e.g., 707): 101

===== CUSTOMER #101 BOOKING HISTORY =====

Booking ID: SAFE-BKG-11ab3290 | Show: #101 | Seat: #3 | Amount: \$500.00 | Status: CONFIRMED | Date: 2026-09-01T11:45:09.037889400

Booking ID: SAFE-BKG-8381e179 | Show: #101 | Seat: #9 | Amount: \$500.00 | Status: CONFIRMED | Date: 2026-09-01T11:45:10.596298500

Booking ID: SAFE-BKG-e5433487 | Show: #101 | Seat: #34 | Amount: \$180.00 | Status: CONFIRMED | Date: 2026-09-01T11:45:12.151404600

Booking ID: SAFE-BKG-ed275573 | Show: #101 | Seat: #38 | Amount: \$180.00 | Status: CONFIRMED | Date: 2026-09-01T11:45:13.704069600

Booking ID: SAFE-BKG-d3c1b8e5 | Show: #101 | Seat: #63 | Amount: \$60.00 | Status: CONFIRMED | Date: 2026-09-01T11:45:15.258627300

Booking ID: SAFE-BKG-78f5964e | Show: #101 | Seat: #87 | Amount: \$60.00 | Status: CONFIRMED | Date: 2026-09-01T11:45:16.814761300

Booking Success

BOOKING CONFIRMED!

✓ Booking ID: SAFE-BKG-dee4098b | Seat #3 (VIP) | Amount: \$500.00

✓ Booking ID: SAFE-BKG-0601c8f | Seat #9 (VIP) | Amount: \$500.00

✓ Booking ID: SAFE-BKG-8d886776 | Seat #34 (Premium) | Amount: \$180.00

✓ Booking ID: SAFE-BKG-3a850000 | Seat #38 (Premium) | Amount: \$180.00

✓ Booking ID: SAFE-BKG-4b5fcd6a | Seat #63 (Regular) | Amount: \$60.00

✓ Booking ID: SAFE-BKG-a77e3cc6 | Seat #87 (Regular) | Amount: \$60.00

OK

YOUR BOOKING

Customer ID: 101

Customer Name: John Doe

Selected Seats

(No seats selected)

TOTAL AMOUNT \$0.00

Payment: ☒ Simulate Success ☐ Simulate Failure

BOOK SELECTED SEATS

Clear Selection

PROCESSING PAYMENT... (State: RESERVED)

CONCURRENCY OBSERVATORY

Ready for stress testing...  
Select 50 or 100 users below.

Run 50-User TestRun 100-User Test

## 17. Testing and Results

| Requirement / Test     | Observed Result                               | Status         |
|------------------------|-----------------------------------------------|----------------|
| OOP / Polymorphism     | Regular, Premium and VIP pricing demonstrated | PASS           |
| Collections / Iterator | Map/List and explicit Iterator traversal      | PASS           |
| 50-user race condition | 19 successes, 18 duplicates                   | BUG REPRODUCED |

|                        |                                     |      |
|------------------------|-------------------------------------|------|
| 50-user ReentrantLock  | 1 success, 0 duplicates             | PASS |
| 100-user ReentrantLock | 1 success, 0 duplicates             | PASS |
| Payment rollback       | Seat restored to AVAILABLE          | PASS |
| Custom exceptions      | All four caught                     | PASS |
| JDBC persistence       | Booking inserted and queried        | PASS |
| Cancellation           | Database CANCELLED; seat restored   | PASS |
| Customer history       | Customer-specific records retrieved | PASS |
| Audit logging          | Audit log maintained                | PASS |
| Invariant verification | All stated invariants satisfied     | PASS |

The final automated verification suite completed with OVERALL RESULT: PASS. Compilation completed with zero errors.

## 18. Scalability and Distributed Deployment

The current ReentrantLock protects state within a single JVM instance. If the application is deployed across multiple application servers, JVM locks cannot coordinate access between servers. A production design would therefore move the final concurrency guarantee to a shared coordination mechanism, typically database-level row locking/transactions or an appropriate distributed locking architecture. The current assignment requires this as discussion rather than implementation.

## 19. Security Considerations

The project uses simulated payment rather than storing real payment credentials. In a production system, sensitive payment data should not be stored directly by the booking application unless required; payment processing should use a compliant payment provider, tokenization, strong access controls, encryption in transit and at rest, and secure secret management. PreparedStatement also reduces SQL injection risk for database operations.

## 20. Professional and Real-World Impact

A double-booking defect in a production ticketing platform can cause financial losses through refunds and chargebacks, operational costs, customer complaints, and damage to trust. From an engineering perspective, concurrency paths must be tested deliberately because ordinary sequential testing may never reveal these failures. The project therefore treats the race condition as a reproducible test case rather than assuming that apparently correct sequential code is safe under load.

## 21. Engineering Decision and Trade-Offs

| Approach                | Advantages                                          | Limitations                                           |
|-------------------------|-----------------------------------------------------|-------------------------------------------------------|
| Unsynchronized          | Simple; useful for reproducing the bug              | Unsafe; allows duplicate allocations                  |
| synchronized            | Simple and familiar Java mechanism                  | Can cause broad contention depending on locking scope |
| ReentrantLock           | Explicit control; timed acquisition; per-show scope | Still limited to one JVM                              |
| ConcurrentHashMap alone | Concurrent map operations                           | Does not make a multi-step booking transaction atomic |

ReentrantLock was selected because it directly protects the critical booking transaction, supports timed acquisition for `BookingTimeoutException`, and allows per-show contention rather than globally blocking all shows.

## 22. Tools and Technologies

| Tool / Technology | Use                                                                 |
|-------------------|---------------------------------------------------------------------|
| Java / JDK        | Application implementation, OOP, threads, exceptions, Swing and I/O |
| SQLite            | Relational persistence                                              |
| JDBC              | Java-database connectivity                                          |
| PreparedStatement | Parameterized SQL                                                   |
| Swing/AWT         | Interactive desktop dashboard                                       |

|                                       |                                                          |
|---------------------------------------|----------------------------------------------------------|
| Git                                   | Version control / source management                      |
| Antigravity / AI-assisted development | Development assistance and project generation/refinement |
| IDE / Terminal                        | Compilation, execution and verification                  |

### 23. Limitations

- Payment processing is simulated rather than connected to a real gateway.
- ReentrantLock provides coordination only within a single JVM.
- The multi-seat GUI selection is a user-interface enhancement; the core correctness guarantee remains centered on the individual thread-safe booking operation.
- The database is SQLite for demonstration and can be replaced by MySQL/PostgreSQL for a multi-user production deployment.

### 24. Student Reflection

This project provided practical understanding of a concurrency problem that is easy to overlook when writing sequential Java programs. The most important learning was that checking a condition and then changing shared state is not automatically safe simply because each individual statement is correct. The race-condition test made the problem observable, while the ReentrantLock test demonstrated how synchronization changes the result.

I also learned how OOP, collections, exception handling, concurrency, persistence and UI design interact in one application. In particular, the payment rollback scenario showed why failure handling must be designed as part of the transaction rather than treated as an afterthought.

With additional time, the system could be extended with database-level concurrency control for multiple application servers, stronger transaction coordination across inventory and persistence, automated integration tests, and a richer production-style frontend. These improvements were deliberately kept outside the current implementation scope to preserve the correctness of the required Java booking core.

## 25. Conclusion

SeatLock successfully implements a concurrent ticket reservation engine in Java and demonstrates the complete path from domain modeling to persistence and validation. The unsynchronized implementation reproduced the target production bug, with 19 customers successfully booking the same seat and 18 duplicate allocations in the 50-user test. After introducing a per-show ReentrantLock, the same 50-user test produced exactly one successful booking and zero duplicates; the 100-user stress test also produced one successful booking and zero duplicates.

The system additionally demonstrates custom checked exceptions, payment rollback, JDBC persistence, cancellation, customer history, audit logging, invariant validation, and an interactive Swing dashboard. The final automated verification suite completed with an overall PASS. The project therefore satisfies the principal functional and non-functional requirements while providing a reusable foundation for future distributed ticketing deployments.

## 26. References

1. Oracle Java Documentation. Java SE API documentation for `java.util.concurrent.locks.ReentrantLock` and related concurrency utilities.
2. Oracle Java Documentation. JDBC API documentation for `Connection`, `PreparedStatement`, `ResultSet` and transaction control.
3. Oracle Java Documentation. Swing and AWT API documentation.
4. SQLite Documentation. SQLite SQL and transaction documentation.
5. Java Platform documentation for Collections Framework, exceptions, threads and I/O.
6. Assignment specification provided by the course faculty: Concurrent Ticket Reservation & Consistency Engine.
7. AI-assisted development: Antigravity/Gemini-assisted code generation and refinement were used during project development; generated code was compiled, executed and manually verified.

## 27. Appendix A — Key Demonstration Script

1. Launch the application using: `java -cp "bin;lib/*" main.Main`
2. Run the unsynchronized race test and point out duplicate bookings.
3. Run the 50-user ReentrantLock test and point out exactly one success and zero duplicates.
4. Run the 100-user ReentrantLock test to demonstrate additional stress validation.
5. Run the payment failure test and show RESERVED/rollback behavior returning the seat to AVAILABLE.
6. Run JDBC persistence/cancellation and show the database state transition.
7. Launch the Swing dashboard, select seats, show dynamic pricing, and perform a successful booking.
8. Use the failure simulation to demonstrate rollback from the GUI.
9. Finish with the Full Verification Suite and show OVERALL RESULT: PASS.

## 28. Appendix B — Important Viva Definitions

| Term              | One-line explanation                                                                                  |
|-------------------|-------------------------------------------------------------------------------------------------------|
| JDBC              | Java Database Connectivity; the Java API used to communicate with relational databases.               |
| Race condition    | A correctness problem where concurrent threads access shared state in an unsafe order.                |
| ReentrantLock     | An explicit Java lock that allows controlled mutual exclusion and timed acquisition.                  |
| Atomicity         | A booking either completes as a whole or fails without leaving an invalid partial state.              |
| Polymorphism      | Calling an overridden method through a base-class reference so the actual subclass behavior executes. |
| PreparedStatement | A parameterized SQL statement used instead of concatenating values into SQL strings.                  |
| Rollback          | Undoing changes after a transaction failure.                                                          |
| Invariant         | A condition that must remain true for the                                                             |

|          |                                                                       |
|----------|-----------------------------------------------------------------------|
|          | system to be considered consistent.                                   |
| Iterator | An interface-based mechanism for traversing elements of a collection. |